Q15. Product Recommendation as Heuristic Search

> **Question.** Model product recommendation with a heuristic search algorithm: define the heuristic function, start state, goal state, actions and neighbouring states. Trace a hill-climbing path on a 10-product set and discuss local and global maxima. Handle multiple goals, give a vector transition with correction using A*, and suggest an interesting modification of hill climbing.

1. The idea in plain words

Think of each product as a point described by a few numbers. The shopper is also a point: the product they would love. A recommender starts from a popular product and keeps stepping to a more similar product that is closer to the shopper's taste. That is search. It is quick, but a greedy search can stop on a "pretty good" product when a better one sits one category away.

2. Assumptions

- 10 products in 4 categories (phone, tablet, watch, audio), each rated 0–1 on **affordability, quality, battery, style**.
- The shopper's preference vector is **u = (0.45, 0.80, 0.85, 0.65)**: wants quality and battery, medium price.
- The most popular product, **P1 Basic phone**, is the start state.
- Hill climbing moves only to similar products **in the same category** (a small, safe change). A* may also cross categories.

3. The search model

Element	Definition
Heuristic h(p)	Distance between the product vector and u: h(p) = ‖p − u‖. Lower is better, 0 is a perfect match.
Start state	P1 Basic phone (the most popular item)
Goal state	The product with the lowest h, or any product with h ≤ ε
Actions	Move to a similar product (swap to a nearby model; for A*, also to another category)
Neighbours	Hill climbing: other products of the same category. A*: the 3 most similar products overall.
Path cost g (A*)	Feature distance travelled, i.e. how big a jump the shopper must accept

The real h values for u: P6 0.21, P5 0.23, P3 0.28, P4 0.34, P10 0.50, P8 0.61, P2 0.66, P7 0.67, P9 0.71, P1 0.89. The **global best is P6 Pro tablet (h = 0.212)**.

4. Step-by-step

1. Compute h for the current product and list its same-category neighbours.
2. Pick the neighbour with the lowest h (steepest ascent in utility).
3. If it is better, move there and repeat. If not, stop and recommend the current product.

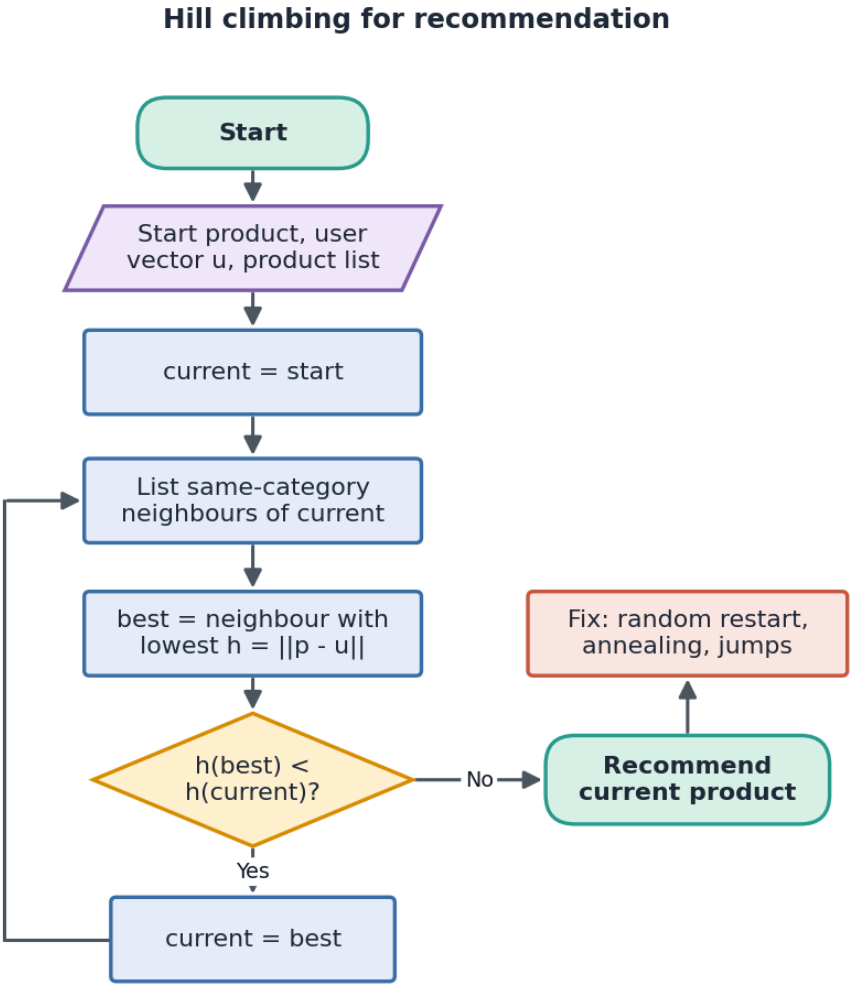


Figure 1. Flowchart of hill climbing for recommendation.

5. Pseudocode

```
function HillClimbRecommend(start, u):
    current ← start
    loop:
        best ← neighbour of current with the lowest h(p) = ||p − u||
        if h(best) ≥ h(current): return current          # local maximum of utility
        current ← best

function AStar(start, goal):
    OPEN ← {start with f = 0};  g(start) ← 0
    while OPEN not empty:
        n ← node in OPEN with lowest f = g(n) + h(n)     # h(n) = ||p_n − p_goal||
        if n = goal: return path
        for each neighbour m of n: g(m) ← g(n) + ||p_n − p_m||; add m to OPEN

function Correct(u, liked, skipped):                      # Rocchio style
    return u + α·(p_liked − u) − β·(p_skipped − u)
```

6. Hill-climbing traversal (worked example)

Step	Product	h	What happened
0	P1 Basic phone	0.890	start
1	P3 Mid phone	0.278	best phone, a big gain; no other phone is closer, so **stop**

The search stops at **P3 (h = 0.278)** while the true best is P6 Pro tablet (0.212).

7. Local maxima and the global maximum

The global maximum is **P6 Pro tablet**. The real run found three **local maxima**: **P3 Mid phone, P8 Smart watch and P10 Earbuds**. Each is the best product inside its own category, so no same-category move improves it. Hill climbing reached the global best from only **3 of 10** start products (the tablets). The cause is the narrow neighbourhood: the better product is one category away and cannot be seen.

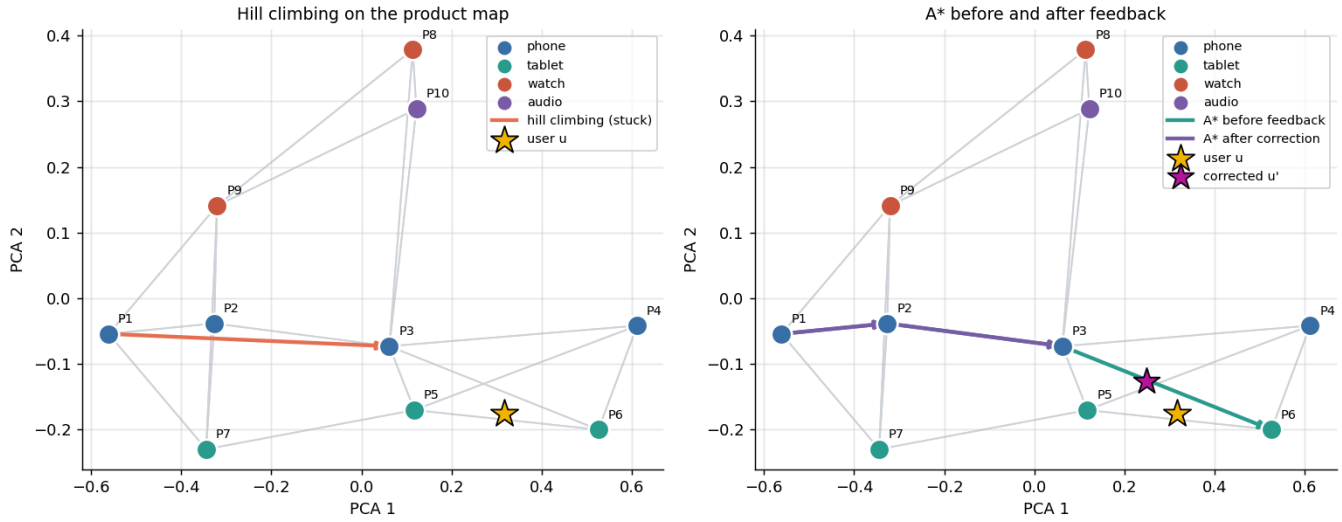


Figure 2. Products on a 2-D PCA map. Left: hill climbing stalls at P3. Right: the A* routes before and after feedback. The stars are u and the corrected u'.

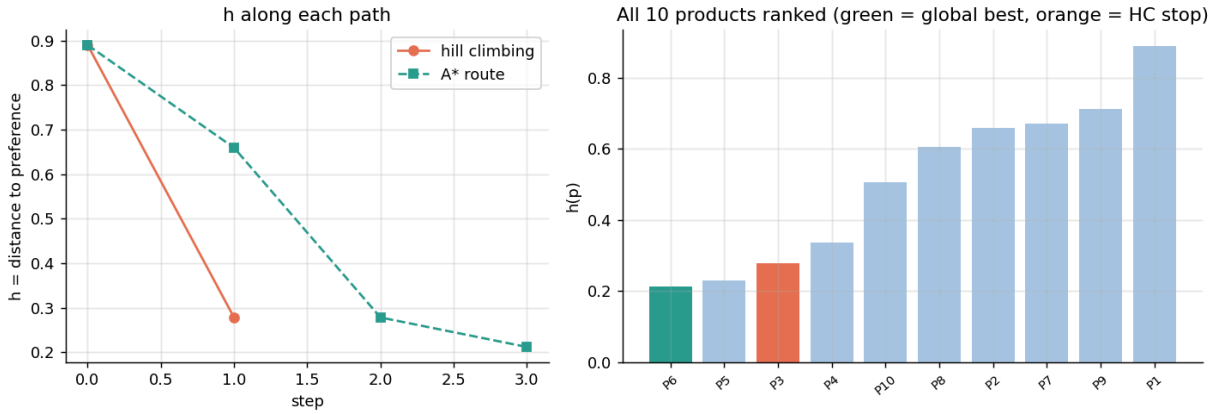


Figure 3. h along the paths, and all products ranked by h.

8. Handling multiple goals

Add a second shopper (a bargain hunter, u_B = (0.85, 0.40, 0.50, 0.40)). Their best product is P9. Three options:

- **Weighted sum:** score = $0.7 \cdot h_A + 0.3 \cdot h_B$. Top 3 in the run: **P5, P3, P6**.

- **Pareto set:** keep every product that no other beats on both goals. Result: **P2, P3, P5, P6, P9**. Show these as choices, not answers.
- **Top-k or multi-start:** run the search once per goal and merge the lists.

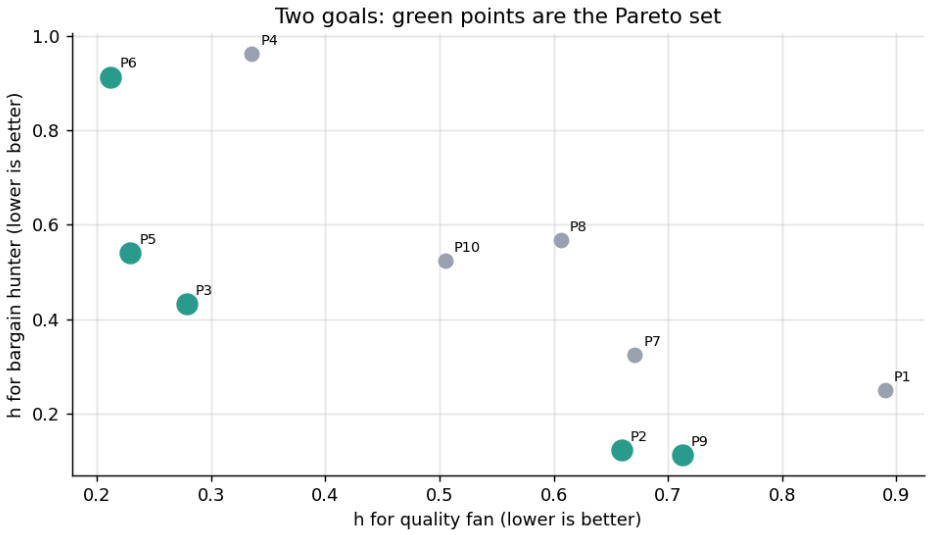


Figure 4. Two goals: green points form the Pareto set.

9. Vector transition with correction using A*

A* uses f = g + h. Here g is the feature distance already travelled and h is the straight-line distance to the goal. It can cross categories, so it finds the tablet.

- **Before feedback** (goal P6): **P1 → P2 → P3 → P6**, with g = 1.112.
- **Feedback:** the shopper likes P3 and skips P5. The vector is corrected with α = 0.5, β = 0.3: $u' = u + 0.5(p_{P3} - u) - 0.3(p_{P5} - u)$ = **(0.50, 0.79, 0.77, 0.62)**.
- **After correction** the nearest product is now P3, so A* gives **P1 → P2 → P3**, with g = 0.630.

The target vector moved toward what the shopper actually liked, and the route got shorter and safer.

10. Interesting modifications to hill climbing

Idea	Result in this run
Random restart (4 runs)	Ends at P6, h = 0.212, the global best
Annealing + serendipity jumps (accept a worse product sometimes, with a 20% chance of a random jump; cooling lowers the risk)	17 of 20 runs find P6 from the same start that traps plain hill climbing
Tabu list	Remember seen products so the search never loops back
Humanised moves	Mood or season changes h; a deliberate "surprise at the critical point" suggestion near the end keeps it fresh

Plain hill climbing: 3 of 10 starts succeed. Restarts and annealing with jumps repair the gap, because they can leave a local maximum.

11. Code and real run

`q15_code.py` defines the 10 products, h, hill climbing, A*, the correction, restarts and annealing, and draws every chart.



Figure 5. Hill climbing and A* functions (lines 59-88) and the real terminal output.

12. Summary

> Recommendation fits heuristic search: product = state, h = distance to the shopper's taste, neighbours = similar products. Hill climbing is fast but failed to reach the best product from 7 of 10 starts. A* with a corrected preference vector plans a smoother route, and restarts, annealing and serendipity jumps reach the global best.

Files in this folder: `q15_code.py` (source), `q15_code.ipynb` (Colab/Jupyter version, already run), `q15_output.txt` (console output), `images/` (all figures).